

Ders 1 Lab Çalışma Özeti

Ders 1 Lab Çalışma Özeti

Genel Konular

- Dersin işleniş mantığı ve çalışma yaklaşımı
 - Kayıt, ilk uygulama saatinde dersin nasıl takip edilmesi gerektiğini ve uygulamaların öğrenme sürecindeki yerini açıklar.
 - Veri yapıları ve algoritmalar dersinde konuların yalnızca dinlenerek değil, doğrudan kod yazarak ve problem çözerek öğrenilmesi gerektiği vurgulanır.
- C diliyle algoritmik çözüm üretme beklentisi
 - Kodlama gerektiren çalışmalarda C dilinin kullanılacağı, C++ veya C# gibi farklı dillerle hazırlanmış çözümlerin dersin beklentisini karşılamayacağı belirtilir.
 - Hazır kütüphane fonksiyonlarıyla sonucu doğrudan almak yerine, istenen veri yapısı veya algoritmanın öğrencinin kendi çözümüyle kurulması gerektiği açıklanır.
- Ders kaynakları ve teorik arka plan
 - Dersin slaytlarının ana kaynak olduğu, bunun yanında Levitin, Sedgewick ve Cormen kitaplarından yararlanıldığı anlatılır.
 - Kaynak kitapların özellikle algoritma analizi, veri yapısı kavramları ve teorik açıklamalar için destekleyici olduğu belirtilir.
- Dönem konu haritası
 - Algoritmik problem çözme, algoritma analizi ve Big-O notasyonu ile başlanacağı ifade edilir.
 - Listeler ve bağlı listeler, stack ve queue, ağaçlar, binary tree, binary search tree, heap ve heap sort gibi temel veri yapılarının işleneceği belirtilir.
 - Arama algoritmaları, string içinde substring arama, greedy yaklaşım, divide and conquer, sıralama algoritmaları ve graf konularının ders kapsamına dahil olduğu açıklanır.

Hocanın Özellikle Vurguladığı Kısımlar

- Veri yapıları ve algoritmalar dersi uygulama yapmadan öğrenilemez.
 - Konuların kalıcı hale gelmesi için öğrencinin kodu kendi yazması, hata yapması ve çözümü adım adım kurması gerekir.
- Hazır çözüm kullanmak dersin amacına aykırıdır.
 - Örneğin bağlı listeyi sıralamak için hazır bir kütüphane çağırmak yerine, beklenen algoritmik düzenin ve veri yapısı işlemlerinin öğrencinin kendisi tarafından tasarlanması gerekir.
- Ders boyunca kavram karmaşasını azaltmak için bazı terimlerin İngilizce karşılıkları kullanılacaktır.
 - Stack, queue ve heap terimleri Türkçede benzer biçimlerde çevrilebildiği için bu yapıların birbirinden net ayrılması önemlidir.
- Algoritma analizi dersin temel eksenlerinden biridir.
 - Bir çözümün yalnızca doğru çalışması değil, başarımının ve karmaşıklığının da değerlendirilebilmesi gerekir.

Kısa Tekrar Notları

- Kodlama çalışmalarında C dili kullanılmalıdır.
- Hazır kütüphane kullanmak yerine veri yapısı ve algoritma mantığı elle kurulmalıdır.

- Temel kaynaklar Levitin, Sedgewick ve Cormen kitaplarıdır.
- İlk konular algoritmik problem çözme, algoritma analizi, karmaşıklık ve Big-O notasyondur.
- Dersin ana veri yapıları listeler, stack, queue, ağaçlar, heap ve graflardır.
- Dersin ana algoritmik yaklaşımları arama, sıralama, greedy ve divide and conquer başlıkları etrafında ilerler.

Detaylı Açıklamalar (Daha Fazla Detay İsteyenler İçin)

Bu kayıt, doğrudan yeni bir teknik konuyu ayrıntılı biçimde işlemekten çok, Veri Yapıları ve Algoritmalar dersinin nasıl çalışılması gerektiğini ve dönem boyunca hangi kavramların izleneceğini açıklar. Dersin merkezinde, verilen bir problemi uygun veri yapısını seçerek ve kendi algoritmik çözümünü kurarak çözme becerisi vardır. Bu nedenle kodlama çalışmalarında hazır kütüphanelerle sonucu almak yerine, bağlantıların, ekleme-silme adımlarının, sıralama veya arama mantığının öğrenci tarafından tasarlanması beklenir. C dili bu uygulamalar için temel dil olarak kullanılır; amaç yalnızca çalışan bir program elde etmek değil, veri yapısının bellekte ve algoritma akışında nasıl yönetildiğini görebilmektir.

Dersin teorik çerçevesi algoritmik problem çözme ve algoritma analiziyle başlar. Bir algoritmanın doğru sonuç üretmesi yeterli değildir; çalışma süresi, kullanılan işlem sayısı ve girdi büyüdükçe davranışının nasıl değiştiği de incelenmelidir. Bu noktada karmaşıklık analizi ve Big-O notasyonu, sonraki haftalarda işlenecek arama, sıralama ve veri yapısı uygulamalarını değerlendirmek için temel araçlar olarak tanıtılır.

Dönem akışında önce listeler ve bağlı listeler gibi doğrusal yapılar, ardından stack ve queue gibi erişim kuralı belirgin yapılar ele alınır. Daha sonra ağaçlar, binary tree, binary search tree, heap ve heap sort gibi hiyerarşik veya öncelik temelli yapılar işlenir. İlerleyen konularda string arama, greedy yaklaşım, divide and conquer, sıralama algoritmaları ve graf veri yapısı yer alır. Graf bölümünde breadth-first search ve depth-first search gibi dolaşma yöntemleri ile minimum spanning tree ve shortest path gibi problemlere geçiş yapılması hedeflenir.